

SYNOPSIS
on
Overhead Vision-Based Autonomous Robot for Fault
Detection and Supervised Repair

Submitted In Partial Fulfillment of Requirements for the Award of the Degree
of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION

ENGINEERING

Submitted to

DEENBANDHU CHHOTU RAM UNIVERSITY OF
SCIENCE AND TECHNOLOGY, MURTHAL, SONEPAT

By

Parveen (23001003090)

Pritam (23001003094)

Rishi (23001003100)

Rohit Shankar (23001003103)

Under the Supervision of

Dr. Vikas Nehra

(Assistant Professor, ECED, DCRUST, Murthal)



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
(Faculty of Engineering and Technology)
DEENBANDHU CHHOTU RAM UNIVERSITY OF SCIENCE & TECHNOLOGY
MURTHAL, SONEPAT-131039

LIST OF CONTENTS

Sr. No.	Contents	Page No.
1.	INTRODUCTION	2
2.	PROBLEM FORMULATION	3
3.	OBJECTIVES	4
4.	BASIC BLOCK DIAGRAM	5
5.	CIRCUIT DIAGRAM & WORKING	6-7
6.	SOFTWARE/HARDWARE TOOLS TO BE USED	8
7.	COMPONENTS	9
8.	PLAN OF EXECUTION	10
9.	APPLICATIONS	11
10.	ESTIMATED COST	12
11.	REFERENCES	13

INTRODUCTION

Modern industrial environments increasingly demand automated solutions for routine inspection and maintenance tasks - particularly in areas that are hazardous, repetitive, or difficult for human operators to access frequently. Autonomous ground robots, guided by overhead computer vision systems, present a practical and scalable solution to this challenge.

This project proposes the design and implementation of a **SCADA-driven autonomous patrol robot** capable of navigating a defined arena, identifying faults at target stations, and executing a supervised repair sequence upon human authorization. The system mirrors the architecture of real industrial SCADA (Supervisory Control and Data Acquisition) systems, where field-level devices report to a central intelligence layer that a human operator monitors and controls.

The distinguishing feature of this system is the **Human-in-the-Loop (HIL)** design philosophy - the robot operates autonomously during navigation and inspection, but critical actions (repair authorization) require explicit human approval before proceeding. This ensures safety, accountability, and operator awareness at all times.

The system is built around a low-cost hardware platform, using an ESP32-C6 microcontroller as the robot's onboard brain, paired with a laptop that handles all high-level intelligence. Rather than relying on expensive onboard sensors like LiDAR or SLAM, the robot's position is tracked entirely through an overhead camera mounted above the arena. This camera feeds a continuous video stream to the laptop, where computer vision algorithms process each frame to determine the robot's location and heading in real time. This design choice keeps the hardware budget minimal while still achieving reliable and accurate navigation.

On the software side, the system leverages **ROS2** and its **Nav2** navigation framework to handle path planning, obstacle avoidance, and motion control. A multi-threaded SCADA hub running on the laptop manages vision processing, navigation control, and the operator dashboard simultaneously. When the robot reaches a target station, an onboard ESP32-CAM module performs a close-range inspection by capturing and decoding a fault code on the station's face. The result is reported back to the hub, and if a fault is found, the operator is immediately notified through the dashboard and prompted to make a decision before any repair action is taken.

PROBLEM FORMULATION

Manual inspection of equipment in industrial or warehouse environments is time-consuming, error-prone, and in some scenarios hazardous. Human inspectors must physically visit each station, assess its health, and escalate faults - a process that scales poorly as the number of stations grows.

In large facilities with dozens of inspection points, this becomes a significant drain on manpower and time. A single missed fault or a delayed response can lead to equipment failure, production downtime, or in worst cases, safety incidents. There is a clear need for a system that can handle routine inspection autonomously, flag issues immediately, and do so without putting human operators at risk.

The core problem addressed in this project is:

How can a low-cost autonomous robotic system reliably patrol multiple inspection points, detect faults using onboard sensing, and notify a human operator for authorization - without requiring expensive onboard computation or complex sensor suites?

Constraints include:

- Limited hardware budget
- No reliance on expensive LiDAR or SLAM
- Must operate reliably in a controlled indoor arena
- Human operator must remain in the decision loop for repair actions
- System must be demonstrable within a compact arena environment

OBJECTIVES

1. Design and build a differential-drive patrol robot capable of navigating autonomously between multiple target stations within a defined arena.
2. Implement an overhead computer vision pipeline to provide real-time positional tracking of the robot and target stations without onboard sensors.
3. Integrate a close-range local inspection module on the robot to detect fault codes at each station.
4. Develop a multi-threaded SCADA software hub on a laptop that manages vision, control, and GUI concurrently.
5. Implement a Human-in-the-Loop authorization gate - the robot pauses upon fault detection and awaits operator approval before initiating the repair sequence.
6. Provide clear feedback during the repair event for operator awareness.
7. Demonstrate full end-to-end operation:
patrol → detect → alert → authorize → repair → resume.

BASIC BLOCK DIAGRAM

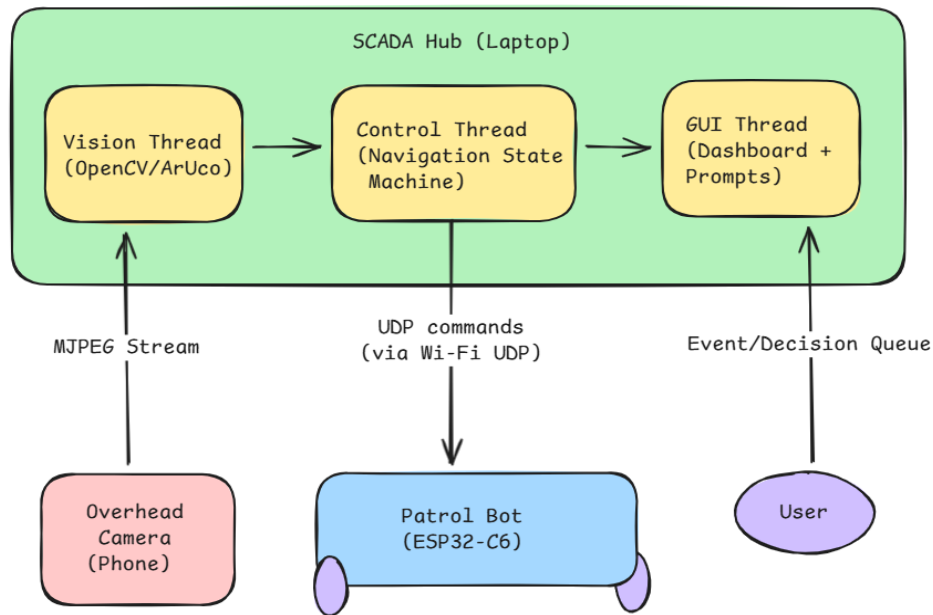


Figure 1. Block Diagram

The system is inspired by SCADA architecture, where a central laptop acts as the brain of the entire operation. All the thinking, decision-making, and coordination happens here, while the robot on the ground simply does what it is told.

Three things connect to this central hub. The phone camera mounted overhead keeps sending a live video feed of the arena to the laptop. The patrol bot receives movement instructions from the laptop and acts on them. The human operator watches the dashboard and steps in when a decision is needed.

Inside the laptop, three parts work together continuously. The Vision Thread watches the camera feed and figures out where the robot is and what is happening in the arena at any given moment. It keeps passing this information to the Control Thread. The Control Thread is the core of the system - it decides where the robot should go next, sends driving commands to the bot, and keeps track of what has been inspected and what has not. The GUI Thread handles everything the operator sees - it shows a live map of the arena, updates the status of each station, and pops up a prompt whenever the operator needs to make a call.

The flow of data is straightforward. The camera feeds video to the Vision Thread, which hands off the robot's position to the Control Thread. The Control Thread sends commands to the bot over Wi-Fi, and the bot passes scan instructions down to its onboard camera module. Once the camera module reads the fault code on a station, the result travels back up the same path to the Control Thread. If a fault is found, the Control Thread alerts the GUI, the operator sees a prompt and decides, and that decision comes back to the Control Thread to act on.

At no point does the robot make any decision on its own. Every meaningful action either originates from or is approved by the central hub, keeping the human operator aware and in control throughout the mission.

CIRCUIT DIAGRAM & WORKING

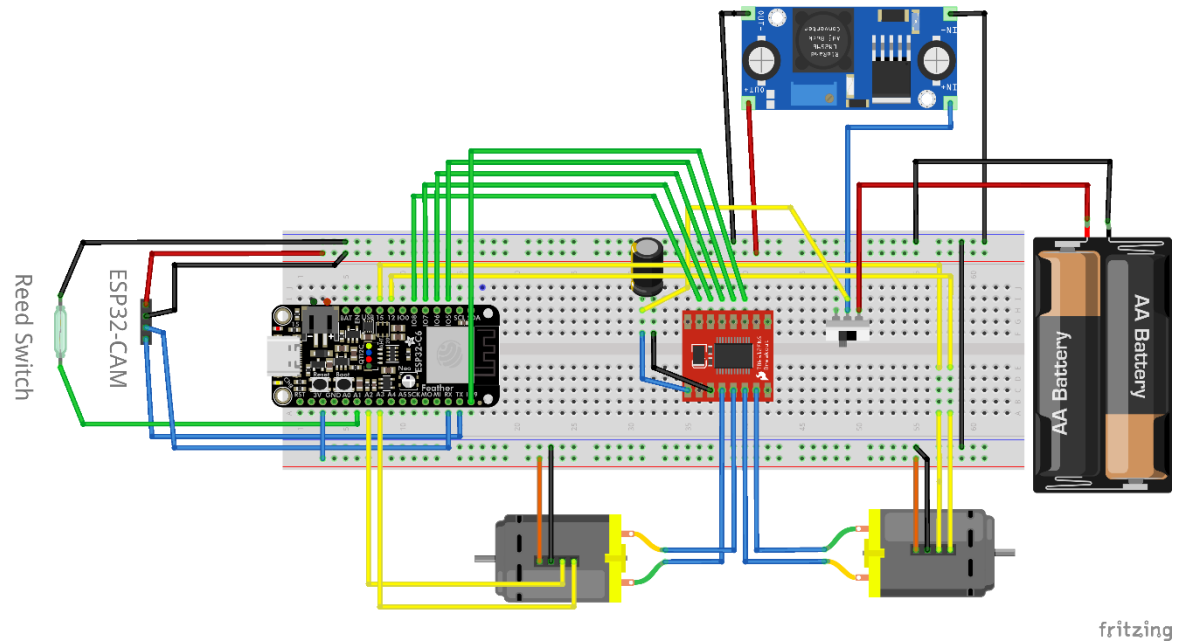


Figure 2. Circuit Diagram

When the system starts, the laptop connects to the phone's Wi-Fi hotspot and launches three concurrent threads- the Vision Thread, Control Thread, and GUI Thread. The ESP32-C6 on the bot connects to the same network and opens a UDP socket, ready to receive motor commands. All communication between the laptop and the bot happens over Wi-Fi UDP, keeping the system lightweight and wireless.

Before navigation begins, the Vision Thread performs a one-time arena calibration using OpenCV. It detects ArUco markers at the four corners of the arena and corrects the camera's angled view into a clean top-down map. It then records the positions of all target boxes.

From this point, the camera continuously tracks the bot's rooftop ArUco marker to determine its position and heading, which is combined with wheel encoder data to maintain a smooth and accurate location estimate throughout the mission.

Once the map is ready, the Control Thread begins the patrol. The entire navigation stack runs on ROS2, which handles path planning, obstacle avoidance, and velocity control through its Nav2 framework. ROS2 also manages communication between all the software components on the laptop, keeping vision, control, and the GUI in sync.

Velocity commands generated by Nav2 are sent to the bot over UDP, where the ESP32-C6 converts them into motor signals. As the bot nears a target box, a VL53L0X distance sensor takes over and brings it to a precise stop right in front of the box.

At each box, the ESP32-C6 triggers the onboard ESP32-CAM over UART, which captures an image of the box's side code and decodes it. The result comes back to the Control Thread via UDP. If the box is healthy, the bot moves on. If a fault is detected, the bot halts and the GUI raises an alert for the operator.

The operator either skips or authorizes the repair. If authorized, the bot drives forward slowly until its front-mounted reed switch is triggered by the magnet embedded in the box. Once triggered, the fault is considered resolved, the world map is updated, and the GUI reflects the repaired status on the dashboard.

This cycle - navigate, dock, inspect, authorize, repair - repeats until all boxes are done, after which the bot returns home and the mission is complete.

SOFTWARE/HARDWARE TOOLS TO BE USED

1. Software

- Python 3.x
- OpenCV
- ROS2
- Nav2
- Arduino IDE
- ESP-IDF

2. Hardware

- Multimeter
- Soldering Kit
- Breadboard
- Perf boards

COMPONENTS

Table 1. Components List

Sr. No.	Name	Value	Specifications	Quantity
1.	ESP32-C6	3.3V logic, 5V input	RISC-V 160MHz, Wi-Fi-6, 512KB SRAM	1
2.	ESP32-CAM	3.3V logic, 5V input	OV3660, 2MP, Wi-Fi, UART	1
3.	ESP32-CAM USB Module	5V USB	Micro USB programmer for ESP32-CAM	1
4.	Motor Driver	2.7-10.8V, 1.5A/ch	DRV8833, TB6612FNG	1
5.	N20 Encoder motors	6V, 200 rpm	Quadrature encoder, DC geared motor	2
6.	TOF sensor	2.8-5V, ~2m range	VL53L0X, 940nm laser, I2C interface, ~50Hz	1
7.	Li-Ion cell	3.7V, 2600mAh	18650 Li-Ion Cell, NMC, 3C discharge, no BMS	2
8.	Mini 360 Buck Converter	Input 4.5–28V, Output 5V	1.8A max output, adjustable	1
9.	Reed Switch (NO)	Normally Open	Glass tube, closes under magnetic field	1
10.	Neodymium magnets	N52 grade	20 x 5 mm disc	5
11.	Acrylic Sheet	A3, A4 size	3mm thick, UV resistant	1

PLAN OF EXECUTION

Table 2. Plan of Execution

Sr. No.	Task	Date
1.	Project Topic Finalization	20 March 2026
2.	Synopsis Finalization	22 April 2026
3.	Literature Survey	28 July 2026
4.	Components Purchase	7 August 2026
5.	Chassis Assembly and Wiring	25 August 2026
6.	Firmware Development and Testing	20 September 2026
7.	Vision and Navigation Pipeline	15 October 2026
8.	Full System Integration and Testing	10 November 2026
9.	Report Finalization	End of Semester

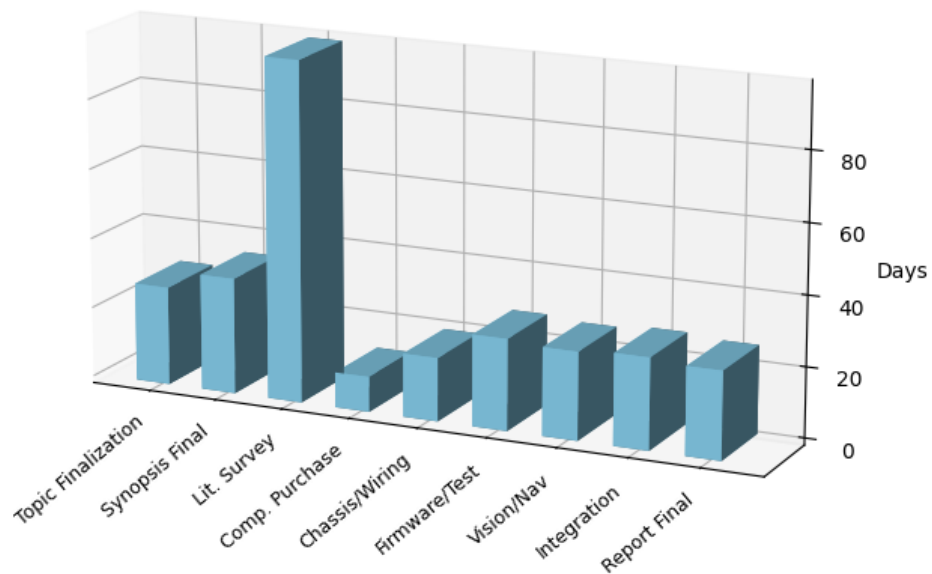


Figure 3. Plan of Execution

APPLICATIONS

- **Industrial Equipment Inspection** : automated patrol of machinery or sensor nodes in a factory floor, flagging anomalies for human review
- **Warehouse Inventory Monitoring** : scanning shelf markers to detect misplaced or faulty items without manual walkthroughs
- **Hazardous Environment Inspection** : remote-supervised operation in areas with chemical, thermal, or radiation risk where human entry is minimized
- **Smart Infrastructure Monitoring** : periodic inspection of utility stations, electrical panels, or pipeline valves in large facilities
- **Educational Demonstration** : practical showcase of SCADA architecture, computer vision-based localization, and human-robot interaction for academic and training purposes
- **Security Patrolling** : systematic scanning of defined zones with human-authorized response on detection

ESTIMATED COST

Table 3. Bill of Materials

Sr. No.	Name	Quantity	Price (in ₹) per unit
1.	ESP32-C6	1	550
2.	ESP32-CAM	1	709
3.	ESP32-CAM USB Module	1	98
4.	Motor Driver	1	72
5.	N20 Encoder motors	2	558
6.	TOF sensor	1	114
7.	Li-Ion cell	2	249
8.	Mini 360 Buck Converter	1	31
9.	Reed Switch (NO)	1	40
10.	Neodymium magnets	5	30
11.	Acrylic Sheet	1	200
Total Cost			3578

REFERENCES

- **Garrido-Jurado, S., et al.** - "Automatic generation and detection of highly reliable fiducial markers under occlusion", Pattern Recognition, 2014. (ArUco marker system)
- **OpenCV Documentation** - ArUco Marker Detection:
https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html
- **Espressif Systems** - ESP32-C6 Technical Reference Manual:
<https://www.espressif.com/en/support/documents/technical-documents>
- **Espressif Systems** - ESP32-CAM Datasheet and Application Notes
- **Texas Instruments** - DRV8833 Dual H-Bridge Motor Driver Datasheet (SLVSA90)
- **Bradski, G.** - "The OpenCV Library", Dr. Dobb's Journal of Software Tools, 2000
- **Mohan, R., et al.** - "SCADA Systems and their Applications in Modern Industry", International Journal of Engineering Research, 2019